

Numpy

July 8, 2026

PAO25_25_06_Python_NumPy

July 06, 2026

PAO26-26 - Python 101, NumPy

Kevin Xavier Suasnavas Villarreal

Link a mi GitHub

1 1 NumPy ndarray

NumPy (*Numerical Python*) es la biblioteca estándar más utilizada para realizar cálculos numéricos y científicos en Python.

Su principal estructura de datos es el **ndarray**, un arreglo multidimensional altamente eficiente que permite almacenar y procesar grandes cantidades de datos de manera mucho más rápida que las listas tradicionales de Python.

Entre sus principales características se encuentran:

- Manipulación eficiente de arreglos multidimensionales.
- Alto rendimiento gracias a que está implementada en lenguaje C.
- Funciones para álgebra lineal y operaciones matemáticas.
- Herramientas para estadística descriptiva.
- Funciones para el procesamiento y transformación de datos, como **np.where()**.

1.1 Importar la biblioteca NumPy

Antes de utilizar cualquier función de NumPy es necesario importar la biblioteca. Por convención, se utiliza el alias **np**, lo que permite escribir un código más corto y legible.

```
[1]: # =====  
# Importar la biblioteca NumPy  
# =====  
  
# Importamos la biblioteca NumPy utilizando  
# el alias convencional "np".  
import numpy as np
```

2 1.1 Performance Sample: NumPy vs Plain Python

Una de las principales ventajas de **NumPy** es su velocidad de procesamiento.

Debido a que sus operaciones están implementadas en lenguaje **C**, trabajar con arreglos de NumPy suele ser considerablemente más rápido que realizar las mismas operaciones utilizando listas tradicionales de Python.

En los siguientes ejemplos se comparará el tiempo necesario para multiplicar diez millones de elementos utilizando ambos enfoques.

2.0.1 Ejemplo utilizando listas de Python

En este primer ejemplo se crea una lista con diez millones de números y posteriormente se multiplica cada elemento por dos mediante una comprensión de listas.

La instrucción mágica **%time** de Jupyter Notebook permite medir el tiempo de ejecución del proceso.

```
[3]: # =====  
# Rendimiento utilizando listas de Python  
# =====  
  
# Creamos una lista con diez millones de elementos.  
my_list = list(range(10_000_000))  
  
# Medimos el tiempo que tarda en multiplicar  
# cada elemento por dos.  
%time my_list = [x * 2 for x in my_list]
```

CPU times: total: 1.56 s

Wall time: 1.7 s

2.0.2 Ejemplo utilizando arreglos de NumPy

Ahora se realizará exactamente la misma operación utilizando un **NumPy Array**.

Gracias a la vectorización de operaciones, no es necesario recorrer manualmente el arreglo mediante ciclos.

```
[4]: # =====  
# Rendimiento utilizando NumPy  
# =====  
  
# Creamos un arreglo de NumPy con diez millones  
# de elementos.  
my_arr = np.arange(10_000_000)  
  
# Multiplicamos todos los elementos del arreglo  
# por dos utilizando operaciones vectorizadas.  
%time my_arr2 = my_arr * 2
```

CPU times: total: 46.9 ms
Wall time: 91.6 ms

2.0.3 Conclusión

Los arreglos de **NumPy** ofrecen un rendimiento significativamente superior al de las listas tradicionales de Python cuando se realizan operaciones matemáticas sobre grandes volúmenes de datos.

Esta mejora de rendimiento se debe a que NumPy implementa operaciones vectorizadas y optimizadas en lenguaje **C**, reduciendo el uso de ciclos explícitos y mejorando considerablemente la eficiencia del procesamiento.

3 1.2 Formas de crear ndarrays

Existen diferentes maneras de crear arreglos (**ndarray**) en NumPy.

Dependiendo de la información disponible, un arreglo puede construirse a partir de listas de Python, secuencias numéricas o mediante funciones que generan matrices con valores específicos.

A continuación se presentan las formas más comunes de crear un **ndarray**.

3.0.1 Crear un arreglo bidimensional

Un **ndarray** puede crearse directamente a partir de una lista de listas. Cada lista interna representa una fila del arreglo.

```
[5]: # =====  
# Crear un ndarray bidimensional  
# =====  
  
# Importamos NumPy.  
import numpy as np  
  
# Creamos un arreglo bidimensional.  
array = np.array([  
    [2, 71, 0, 34],  
    [2, 171, -35, 34]  
)  
  
# Mostramos el arreglo.  
print(array)
```

```
[[ 2  71   0  34]  
 [ 2 171 -35  34]]
```

3.0.2 Convertir una lista en un ndarray

También es posible convertir una lista tradicional de Python en un arreglo de NumPy mediante la función **np.array()**.

```
[6]: # =====
# Convertir una lista en un ndarray
# =====

# Lista bidimensional de ejemplo.
shopping_list = [
    ["onions", "carrots", "celery"],
    ["apples", "oranges", "grapes"]
]

# Convertimos la lista en un ndarray.
array = np.array(shopping_list)

# Mostramos el arreglo.
print(array)

# Mostramos su tipo de dato.
print(type(array))
```

```
[['onions' 'carrots' 'celery']
 ['apples' 'oranges' 'grapes']]
<class 'numpy.ndarray'>
```

3.0.3 Crear arreglos desde listas y secuencias

NumPy permite generar arreglos tanto a partir de listas existentes como mediante secuencias numéricas similares a `range()`.

```
[7]: # =====
# Crear arreglos unidimensionales
# =====

# Crear un arreglo a partir de una lista.
print(np.array([100, 10, 1]))

print()

# Crear un arreglo mediante una secuencia.
print(np.arange(2, 10, 2.1))
```

```
[100  10   1]

[2.  4.1 6.2 8.3]
```

3.0.4 Crear arreglos con valores predefinidos

NumPy dispone de varias funciones para generar arreglos con valores iniciales específicos, como ceros, unos, valores constantes o matrices identidad.

```
[8]: # =====
# Crear arreglos con valores fijos
# =====

# Vector de ceros.
print(np.zeros(2))

print()

# Matriz de unos.
print(np.ones((4, 3)))

print()

# Arreglo sin inicializar.
print(np.empty((2, 2, 2)))

print()

# Matriz llena con el valor 2.
print(np.full((5, 2), 2))

print()

# Matriz identidad.
print(np.eye(3))
```

```
[0. 0.]
```

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

```
[[[9.88e-324 3.51e-322]
  [0.00e+000 1.68e-322]]
```

```
[[[9.88e-324 8.45e-322]
  [          nan 1.68e-322]]]
```

```
[[2 2]
 [2 2]
 [2 2]
 [2 2]
 [2 2]]
```

```
[[1. 0. 0.]
 [0. 1. 0.]
```

```
[0. 0. 1.]
```

4 1.3 Obtener información sobre un ndarray

NumPy proporciona numerosos atributos y funciones que permiten conocer las características de un arreglo, como su tamaño, dimensiones, forma y ubicación de determinados elementos.

4.0.1 Obtener el valor máximo y mínimo

Es posible localizar tanto el valor máximo como el mínimo de un arreglo, así como conocer la posición donde se encuentran.

```
[9]: # =====  
# Máximos y mínimos  
# =====  
  
# Creamos un arreglo.  
my_ndarray = np.array([2, 71, 0, 34])  
  
# Mostramos el arreglo.  
print(my_ndarray)  
  
print()  
  
# Índice del valor máximo.  
print(my_ndarray.argmax())  
  
# Valor máximo.  
print(np.amax(my_ndarray))  
  
print()  
  
# Índice del valor mínimo.  
print(my_ndarray.argmin())  
  
# Valor mínimo.  
print(np.amin(my_ndarray))  
  
print()  
  
# Índices de los elementos diferentes de cero.  
print(my_ndarray.nonzero())
```

```
[ 2 71  0 34]
```

```
1
```

```
71
```

2
0

(array([0, 1, 3]),)

4.0.2 Información de un arreglo multidimensional

Los atributos **size**, **shape** y **ndim** permiten conocer el número de elementos, la forma del arreglo y la cantidad de dimensiones que posee.

```
[10]: # =====  
# Información de un arreglo multidimensional  
# =====  
  
# Creamos un arreglo tridimensional.  
my_ndarray = np.full((5, 4, 2), 2)  
  
# Mostramos el arreglo.  
print(my_ndarray)  
  
print()  
  
# Número total de elementos.  
print(my_ndarray.size)  
  
# Forma del arreglo.  
print(my_ndarray.shape)  
  
# Número de dimensiones.  
print(my_ndarray.ndim)
```

```
[[[2 2]  
   [2 2]  
   [2 2]  
   [2 2]]
```

```
[[2 2]  
 [2 2]  
 [2 2]  
 [2 2]]
```

```
[[2 2]  
 [2 2]  
 [2 2]  
 [2 2]]
```

```
[[2 2]  
 [2 2]]
```

```

[2 2]
[2 2]]

[[2 2]
 [2 2]
 [2 2]
 [2 2]]]

40
(5, 4, 2)
3

```

4.0.3 Información de un arreglo unidimensional

Los mismos atributos también pueden utilizarse con arreglos de una sola dimensión.

```

[11]: # =====
# Información de un arreglo unidimensional
# =====

# Creamos un arreglo.
my_ndarray = np.array([2, 71, 0, 34])

# Número de elementos.
print(my_ndarray.size)

# Forma del arreglo.
print(my_ndarray.shape)

# Número de dimensiones.
print(my_ndarray.ndim)

4
(4,)
1

```

4.0.4 Conclusión

NumPy ofrece múltiples formas de crear arreglos adaptándose a diferentes necesidades. Además, proporciona atributos y funciones que permiten conocer rápidamente las características de un **ndarray**, facilitando el procesamiento y análisis de grandes volúmenes de datos de manera eficiente.

4.1 Redimensionar los datos

Una de las ventajas de los **ndarray** es que pueden cambiar su forma sin modificar los datos almacenados.

La función **reshape()** permite reorganizar los elementos de un arreglo siempre que el número total de elementos permanezca constante.

4.1.1 Cambiar la forma de un arreglo

En este ejemplo se crea un arreglo de una fila y ocho columnas para observar posteriormente cómo puede reorganizarse en distintas dimensiones.

```
[12]: # =====
# Crear un arreglo de 1 fila por 8 columnas
# =====

# Creamos un arreglo bidimensional de tamaño 1x8.
my_ndarray = np.array([[0, 71, 21, 19, 213, 412, 111, 98]])

# Mostramos el arreglo.
print(my_ndarray)

# Mostramos su forma.
print(my_ndarray.shape)

# Mostramos el número de dimensiones.
print(my_ndarray.ndim)

[[ 0  71  21  19 213 412 111  98]]
(1, 8)
2
```

4.1.2 Redimensionar a una matriz de 2×4

Con `reshape()` es posible reorganizar los elementos del arreglo en dos filas y cuatro columnas.

```
[13]: # =====
# Redimensionar a 2x4
# =====

# Cambiamos la forma del arreglo.
new_dims = my_ndarray.reshape(2, 4)

# Mostramos el nuevo arreglo.
print(new_dims)

# Mostramos sus dimensiones.
print(new_dims.ndim)

[[ 0  71  21  19]
 [213 412 111  98]]
2
```

4.1.3 Redimensionar a una matriz de 2×4

Con `reshape()` es posible reorganizar los elementos del arreglo en dos filas y cuatro columnas.

```
[14]: # =====
# Redimensionar a 2x4
# =====

# Cambiamos la forma del arreglo.
new_dims = my_ndarray.reshape(2, 4)

# Mostramos el nuevo arreglo.
print(new_dims)

# Mostramos sus dimensiones.
print(new_dims.ndim)
```

```
[[ 0  71  21  19]
 [213 412 111  98]]
2
```

4.1.4 Redimensionar a una matriz de 4×2

También es posible reorganizar los mismos datos en cuatro filas y dos columnas.

```
[15]: # =====
# Redimensionar a 4x2
# =====

new_dims = my_ndarray.reshape(4, 2)

print(new_dims)
print(new_dims.ndim)
```

```
[[ 0  71]
 [ 21  19]
 [213 412]
 [111  98]]
2
```

4.1.5 Convertir un vector columna

Al utilizar **reshape(3,1)**, un arreglo unidimensional puede transformarse en un vector columna.

```
[16]: # =====
# Convertir un vector en columna
# =====

# Creamos un vector columna.
x = np.array([100, 10, 1]).reshape(3, 1)

# Mostramos la forma.
print(x.shape)
```

```
# Mostramos el contenido.  
print(x)
```

```
(3, 1)  
[[100]  
 [ 10]  
 [ 1]]
```

4.1.6 Reducir un arreglo a una dimensión

La función **flatten()** convierte un arreglo multidimensional en un vector de una sola dimensión.

```
[17]: # =====  
# Convertir un arreglo en un vector  
# =====  
  
# Creamos una matriz de 3x3.  
new_dims = np.arange(9).reshape(3, 3)  
  
# Mostramos la matriz original.  
print(new_dims)  
  
print()  
  
# Convertimos la matriz en un vector.  
print(new_dims.flatten())
```

```
[[0 1 2]  
 [3 4 5]  
 [6 7 8]]
```

```
[0 1 2 3 4 5 6 7 8]
```

4.1.7 Inferir automáticamente una dimensión

Al utilizar el valor **-1**, NumPy calcula automáticamente la dimensión necesaria para conservar la cantidad total de elementos.

```
[18]: # =====  
# Inferir automáticamente una dimensión  
# =====  
  
# Creamos una matriz de 4x4.  
new_dims = np.arange(16).reshape(4, 4)  
  
print(new_dims)  
  
print()
```

```
# NumPy calcula automáticamente el número de columnas.
print(new_dims.reshape(2, -1))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]]
```

5 1.4 Manipulación de ndarrays

Además de crear y redimensionar arreglos, NumPy ofrece diversas funciones para modificar su orientación, invertir el orden de los elementos y ordenarlos de forma eficiente.

5.0.1 Intercambiar ejes

La función **swapaxes()** permite intercambiar dos ejes de un arreglo multidimensional.

```
[19]: # =====
# Intercambiar ejes
# =====

# Creamos una matriz de 8 filas y 2 columnas.
new_dims = np.arange(16).reshape(8, 2)

# Mostramos la matriz original.
print(new_dims)

print()

# Intercambiamos filas por columnas.
print(new_dims.swapaxes(0, 1))
```

```
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]
 [12 13]
 [14 15]]
```

```
[[ 0  2  4  6  8 10 12 14]
 [ 1  3  5  7  9 11 13 15]]
```

5.0.2 Invertir un arreglo

La función `np.flip()` invierte el orden de los elementos del arreglo.

Cuando `axis=None`, se invierten todos los ejes del arreglo.

```
[20]: # =====  
# Invertir un arreglo  
# =====  
  
# Creamos una matriz de ejemplo.  
new_dims = np.arange(16).reshape(4, 4)  
  
# Mostramos la matriz original.  
print(new_dims)  
  
print()  
  
# Invertimos completamente el arreglo.  
print(np.flip(new_dims, axis=None))
```

```
[[ 0  1  2  3]  
 [ 4  5  6  7]  
 [ 8  9 10 11]  
 [12 13 14 15]]
```

```
[[15 14 13 12]  
 [11 10  9  8]  
 [ 7  6  5  4]  
 [ 3  2  1  0]]
```

5.0.3 Ordenar un arreglo

Los arreglos de NumPy pueden ordenarse fácilmente utilizando el método `sort()`.

```
[21]: # =====  
# Ordenar un arreglo  
# =====  
  
# Creamos un arreglo desordenado.  
array1 = np.array([10, 2, 9, 17])  
  
# Ordenamos los elementos de menor a mayor.  
array1.sort()  
  
# Mostramos el resultado.  
print(array1)
```

```
[ 2  9 10 17]
```

5.0.4 Conclusión

Las funciones de redimensionamiento y manipulación de **NumPy** permiten reorganizar la información de forma sencilla y eficiente sin necesidad de copiar los datos manualmente. Estas herramientas resultan fundamentales para preparar datos antes de realizar operaciones matemáticas, análisis estadísticos o procesos de aprendizaje automático.

5.0.5 Unir arreglos

NumPy permite combinar varios arreglos de diferentes maneras. Dependiendo de la orientación deseada, los arreglos pueden unirse horizontalmente, verticalmente o mediante concatenación sobre un eje específico.

Unión horizontal y vertical Las funciones `np.hstack()` y `np.vstack()` permiten unir arreglos horizontal y verticalmente, respectivamente.

```
[22]: # =====  
# Unir arreglos horizontal y verticalmente  
# =====  
  
# Creamos dos arreglos unidimensionales.  
a1 = [0, 0, 0, 0, 0, 0]  
a2 = [1, 1, 1, 1, 1, 1]  
  
# Unión horizontal.  
horizontal = np.hstack((a1, a2))  
print(horizontal)  
print(horizontal.shape)  
  
print()  
  
# Unión vertical.  
vertical = np.vstack((a1, a2))  
print(vertical)  
print(vertical.shape)
```

```
[0 0 0 0 0 0 1 1 1 1 1 1]  
(12,)
```

```
[[0 0 0 0 0 0]  
 [1 1 1 1 1 1]]  
(2, 6)
```

Concatenar arreglos La función `np.concatenate()` permite unir arreglos indicando el eje sobre el cual se realizará la operación.

```
[23]: # =====  
# Concatenar arreglos  
# =====
```

```

# Creamos dos matrices.
array1 = np.array([[2, 4],
                   [6, 8]])

array2 = np.array([[3, 5],
                   [7, 9]])

# Concatenamos las matrices por filas.
print(np.concatenate((array1, array2), axis=0))

```

```

[[2 4]
 [6 8]
 [3 5]
 [7 9]]

```

5.0.6 Dividir arreglos

NumPy también permite dividir un arreglo en varias partes utilizando la función `np.array_split()`.

Dividir un arreglo en partes iguales En este ejemplo se divide una matriz en dos y tres partes.

```

[24]: # =====
# Dividir un arreglo
# =====

# Creamos una matriz de 4x4.
array = np.arange(16).reshape(4, 4)

# Mostramos la matriz original.
print(array)

print()

# Dividimos en dos partes iguales por filas.
print(np.array_split(array, 2, axis=0))

print()

# Dividimos en tres partes.
print(np.array_split(array, 3))

```

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]

```

```
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])]
```

```
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11]]), array([[12, 13, 14, 15]])]
```

Dividir utilizando posiciones específicas También es posible indicar exactamente en qué posición debe dividirse el arreglo.

```
[25]: # =====
# Dividir utilizando un índice específico
# =====

# Creamos nuevamente la matriz.
array = np.arange(16).reshape(4, 4)

print(array)

print()

# Dividimos antes de la columna 3.
print(np.array_split(array, [3], axis=1))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

```
[array([[ 0,  1,  2],
        [ 4,  5,  6],
        [ 8,  9, 10],
        [12, 13, 14]]), array([[ 3],
        [ 7],
        [11],
        [15]])]
```

Acceder a los arreglos generados La función `np.array_split()` devuelve una lista de arreglos, por lo que es posible acceder a cada uno mediante índices.

```
[26]: # =====
# Acceder a una división del arreglo
# =====

# Dividimos la matriz.
split = np.array_split(array, [3], axis=1)

# Accedemos a un elemento específico.
```



```
print(split[0][3][0])
```

12

Dividir un arreglo en múltiples posiciones Es posible indicar varios puntos de división tanto para filas como para columnas.

```
[27]: # =====  
# Dividir un arreglo en múltiples posiciones  
# =====  
  
# Creamos una matriz de 8x8.  
array = np.arange(64).reshape(8, 8)  
  
print(array)  
  
print()  
  
# Dividimos por las filas 1 y 3.  
print(np.array_split(array, [1, 3], axis=0))  
  
print()  
  
# Dividimos por las columnas 1 y 3.  
print(np.array_split(array, [1, 3], axis=1))  
  
# Nota:  
# axis = 0 → Filas  
# axis = 1 → Columnas
```

```
[[ 0  1  2  3  4  5  6  7]  
 [ 8  9 10 11 12 13 14 15]  
[16 17 18 19 20 21 22 23]  
[24 25 26 27 28 29 30 31]  
[32 33 34 35 36 37 38 39]  
[40 41 42 43 44 45 46 47]  
[48 49 50 51 52 53 54 55]  
[56 57 58 59 60 61 62 63]]
```

```
[array([[0, 1, 2, 3, 4, 5, 6, 7]]), array([[ 8,  9, 10, 11, 12, 13, 14, 15],  
      [16, 17, 18, 19, 20, 21, 22, 23]]), array([[24, 25, 26, 27, 28, 29, 30,  
31],  
      [32, 33, 34, 35, 36, 37, 38, 39],  
      [40, 41, 42, 43, 44, 45, 46, 47],  
      [48, 49, 50, 51, 52, 53, 54, 55],  
      [56, 57, 58, 59, 60, 61, 62, 63]])]
```

```
[array([[ 0],
```

```
[ 8],
[16],
[24],
[32],
[40],
[48],
[56]]), array([[ 1,  2],
[ 9, 10],
[17, 18],
[25, 26],
[33, 34],
[41, 42],
[49, 50],
[57, 58]]), array([[ 3,  4,  5,  6,  7],
[11, 12, 13, 14, 15],
[19, 20, 21, 22, 23],
[27, 28, 29, 30, 31],
[35, 36, 37, 38, 39],
[43, 44, 45, 46, 47],
[51, 52, 53, 54, 55],
[59, 60, 61, 62, 63]])]
```

6 2 Índices y slicing

Los arreglos de NumPy utilizan un sistema de indexación muy similar al de las listas de Python.

Esto permite acceder fácilmente a elementos individuales o seleccionar subconjuntos del arreglo mediante la técnica conocida como **slicing**.

6.0.1 Acceso mediante índices y slicing

En el siguiente ejemplo se muestra cómo acceder al último elemento de un arreglo y cómo obtener un subconjunto de sus elementos utilizando **slicing**.

```
[28]: # =====
# Índices y slicing
# =====

# Creamos un arreglo.
array = np.arange(10, 16)

# Mostramos el arreglo completo.
print(array)

print()

# Accedemos al último elemento.
print(array[-1])
```

```
print()

# Obtenemos todos los elementos excepto el último.
print(array[:-1])
```

```
[10 11 12 13 14 15]
```

```
15
```

```
[10 11 12 13 14]
```

6.0.2 Conclusión

Las funciones de unión y división de arreglos permiten reorganizar grandes conjuntos de datos de forma sencilla, mientras que el uso de índices y **slicing** facilita acceder únicamente a la información necesaria para cada operación. Estas herramientas constituyen una base fundamental para el procesamiento de datos con NumPy.

6.1 Acceso a elementos en arreglos bidimensionales

En un arreglo de dos dimensiones, los elementos pueden accederse indicando la **fila** y la **columna** donde se encuentran.

La sintaxis más utilizada es:

```
array[filas, columnas]
```

También es posible acceder utilizando índices consecutivos:

```
array[filas][columnas]
```

Ambas formas producen el mismo resultado.

```
[29]: # =====
# Acceso a elementos de un arreglo 2D
# =====

# Creamos una matriz de 4 filas y 4 columnas.
array = np.arange(16).reshape(4, 4)

# Mostramos la matriz completa.
print(array)

print()

# Accedemos al elemento ubicado en la fila 2, columna 2.
print(array[2, 2])

# Forma equivalente utilizando índices consecutivos.
print(array[3][2])
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

10

14

6.2 Acceso a elementos en arreglos multidimensionales

Los arreglos pueden tener más de dos dimensiones.

En estos casos se debe indicar un índice por cada dimensión existente hasta llegar al elemento deseado.

```
[30]: # =====
# Acceso a un arreglo tridimensional
# =====

# Creamos un arreglo de dimensiones 2x3x6.
array = np.arange(36).reshape(2, 3, 6)

# Mostramos el arreglo.
print(array)

print()

# Accedemos a un elemento específico.
print(array[0, 2, 4])

# Forma equivalente utilizando índices consecutivos.
print(array[1][2][4])
```

```
[[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]]
```

```
[[18 19 20 21 22 23]
 [24 25 26 27 28 29]
 [30 31 32 33 34 35]]
```

16

34

6.3 Organización de los datos: Row Major

Por defecto, NumPy almacena los datos siguiendo el orden **Row Major** (orden por filas).

Esto significa que primero se completan todos los elementos de una fila antes de continuar con la siguiente.

```
[31]: # =====
# Organización Row Major (por defecto)
# =====

# Creamos un arreglo unidimensional.
row_major = np.array([1, 2, 3, 4, 5, 6])

# Lo reorganizamos en una matriz de 2x3.
row_major = row_major.reshape(2, 3)

# Mostramos el resultado.
print(row_major)
```

```
[[1 2 3]
 [4 5 6]]
```

6.3.1 Row Major vs Column Major

La función `reshape()` permite indicar el orden en que se reorganizarán los datos.

- `order='C'` → organización por filas (*Row Major*).
- `order='F'` → organización por columnas (*Column Major*).

```
[32]: # =====
# Comparación entre Row Major y Column Major
# =====

# Organización por filas.
row_major = np.array([1, 2, 3, 4, 5, 6])
row_major = row_major.reshape(2, 3, order="C")

print("Row Major")
print(row_major)

print()

# Organización por columnas.
col_major = np.array([1, 2, 3, 4, 5, 6])
col_major = col_major.reshape(2, 3, order="F")

print("Column Major")
print(col_major)
```

Row Major

```
[[1 2 3]
 [4 5 6]]
```

Column Major

```
[[1 3 5]
```

```
[2 4 6]]
```

6.4 Slicing en arreglos multidimensionales

El **slicing** permite seleccionar únicamente una parte del arreglo.

Es posible realizar cortes tanto por filas como por columnas utilizando la misma sintaxis empleada en las listas de Python.

```
[33]: # =====  
# Slicing en una matriz  
# =====  
  
# Creamos una matriz de 9x9.  
array = np.arange(81).reshape(9, 9)  
  
# Mostramos la matriz completa.  
print(array)  
  
print()  
  
# Seleccionamos filas y columnas específicas.  
print(array[1:3, 2::2])
```

```
[[ 0  1  2  3  4  5  6  7  8]  
 [ 9 10 11 12 13 14 15 16 17]  
[18 19 20 21 22 23 24 25 26]  
[27 28 29 30 31 32 33 34 35]  
[36 37 38 39 40 41 42 43 44]  
[45 46 47 48 49 50 51 52 53]  
[54 55 56 57 58 59 60 61 62]  
[63 64 65 66 67 68 69 70 71]  
[72 73 74 75 76 77 78 79 80]]
```

```
[[11 13 15 17]  
 [20 22 24 26]]
```

6.4.1 Seleccionar una fila completa

Utilizando el operador `:` es posible seleccionar todos los elementos de una fila o de una columna.

```
[34]: # =====  
# Obtener una fila completa  
# =====  
  
# Creamos una matriz de 3x3.  
array = np.arange(9).reshape(3, 3)  
  
# Mostramos la matriz.
```

```
print(array)

print()

# Mostramos la primera fila.
print(array[0, :])
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

```
[0 1 2]
```

6.5 Diferencias entre listas y ndarrays

Existe una diferencia importante entre las listas de Python y los arreglos de NumPy.

- En las **listas**, un **slice** devuelve una **copia** de los datos.
- En los **ndarray**, un **slice** devuelve una **referencia (vista)** del arreglo original.

Este comportamiento influye directamente en la modificación de los datos.

```
[35]: # =====
# Slice en listas (devuelve una copia)
# =====

# Creamos una lista.
lista = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Creamos una copia mediante slicing.
my_copy = lista[0:2]

print(my_copy)

# Modificamos la copia.
my_copy[0] = 222

# La lista original permanece igual.
print(lista)

# Solo cambia la copia.
print(my_copy)
```

```
[0, 1]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[222, 1]
```

6.5.1 Conclusión

Los arreglos de NumPy utilizan un sistema de indexación muy similar al de las listas de Python, pero presentan una diferencia importante: los **slices** generan vistas del arreglo original y no copias independientes. Este comportamiento permite trabajar de forma más eficiente con grandes volúmenes de datos, aunque requiere mayor cuidado al modificar los elementos.

6.6 Referencias y copias en ndarrays

A diferencia de las listas de Python, cuando se realiza un **slice** sobre un **ndarray**, NumPy devuelve una **referencia (vista)** del arreglo original y no una copia independiente.

Esto significa que cualquier modificación realizada sobre el nuevo arreglo también afectará al arreglo original.

```
[37]: # =====  
# Slicing devuelve una referencia  
# =====  
  
# Creamos un arreglo de ejemplo.  
array = np.arange(10)  
  
# Obtenemos una vista del arreglo mediante slicing.  
my_copy = array[2:5]  
  
# Mostramos ambos arreglos.  
print(array)  
print(my_copy)  
  
print()  
  
# Modificamos el primer elemento de la vista.  
my_copy[0] = 222  
  
# Como my_copy es una referencia,  
# también cambia el arreglo original.  
print(array)  
print(my_copy)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
[2 3 4]
```

```
[ 0  1 222  3  4  5  6  7  8  9]
```

```
[222  3  4]
```

6.6.1 Crear una copia independiente

Si se desea modificar un subconjunto del arreglo sin afectar al original, es necesario utilizar el método **copy()**, que crea una copia completamente independiente de los datos.


```
[38]: # =====
# Crear una copia independiente
# =====

# Creamos un arreglo.
array = np.arange(10)

# Creamos una copia del slice.
my_copy = array[2:5].copy()

# Modificamos la copia.
my_copy[0] = 222

# El arreglo original permanece sin cambios.
print(array)

# Solo cambia la copia.
print(my_copy)
```

[0 1 2 3 4 5 6 7 8 9]
[222 3 4]

6.6.2 Asignar valores mediante slicing

El slicing también permite modificar varios elementos del arreglo al mismo tiempo, asignando un único valor a todo un rango de posiciones.

```
[39]: # =====
# Asignar un mismo valor a un slice
# =====

# Creamos un arreglo.
array = np.arange(10)

# Mostramos el arreglo original.
print(array)

print()

# Mostramos los tres primeros elementos.
print(array[0:3])

print()

# Reemplazamos los tres primeros valores por -1.
array[0:3] = -1

# Mostramos el arreglo modificado.
```

```
print(array)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
[0 1 2]
```

```
[-1 -1 -1 3 4 5 6 7 8 9]
```

6.7 Slicing condicional (Máscaras)

NumPy permite seleccionar elementos utilizando **máscaras booleanas** (*Boolean Masking*).

Una máscara es un arreglo de valores **True** y **False** que indica qué elementos deben seleccionarse o modificarse.

6.7.1 Seleccionar elementos mediante una máscara

En este ejemplo se seleccionan únicamente los números pares del arreglo utilizando una condición lógica.

```
[40]: # =====  
# Uso de máscaras booleanas  
# =====  
  
# Creamos un arreglo con valores negativos y positivos.  
array = np.arange(-3, 4)  
  
# Mostramos el arreglo.  
print(array)  
  
print()  
  
# Creamos una máscara para identificar los números pares.  
mask = array % 2 == 0  
  
# Mostramos la máscara.  
print(mask)  
  
print()  
  
# Mostramos únicamente los elementos pares.  
print(array[mask])  
  
print()  
  
# Forma equivalente utilizando la condición directamente.  
print(array[array % 2 == 0])  
  
print()
```

```
# Reemplazamos los números pares por cero.
array[mask] = 0

# Mostramos el arreglo modificado.
print(array)
```

```
[-3 -2 -1  0  1  2  3]
```

```
[False  True False  True False  True False]
```

```
[-2  0  2]
```

```
[-2  0  2]
```

```
[-3  0 -1  0  1  0  3]
```

6.7.2 Modificar elementos que cumplen una condición

Las máscaras también permiten modificar directamente los elementos que satisfacen una determinada condición.

```
[41]: # =====
# Reemplazar valores negativos
# =====

# Creamos un arreglo.
array = np.arange(-3, 4)

# Mostramos el arreglo original.
print(array)

print()

# Reemplazamos todos los valores negativos por cero.
array[array < 0] = 0

# Mostramos el resultado.
print(array)
```

```
[-3 -2 -1  0  1  2  3]
```

```
[0 0 0 0 1 2 3]
```

6.7.3 Conclusión

El uso de **slicing**, copias y máscaras booleanas convierte a NumPy en una herramienta muy potente para manipular grandes conjuntos de datos. Gracias a estas operaciones es posible seleccionar, modificar y filtrar información de manera eficiente, evitando el uso de ciclos innecesarios y mejorando

el rendimiento de los programas.

6.7.4 Las máscaras devuelven una copia

Cuando se utiliza una **máscara booleana** para seleccionar elementos de un arreglo, NumPy devuelve una **copia** de dichos elementos y no una referencia al arreglo original.

Por esta razón, las modificaciones realizadas sobre el nuevo arreglo no afectan al arreglo del que provienen los datos.

```
[42]: # =====
# Máscaras booleanas: devuelven una copia
# =====

# Creamos un arreglo con valores desde -3 hasta 3.
array = np.arange(-3, 4)

# Mostramos el arreglo original.
print(array)

print()

# Creamos una máscara para seleccionar únicamente los números pares.
mask = array % 2 == 0

# Mostramos la máscara.
print(mask)

print()

# Creamos un nuevo arreglo utilizando la máscara.
# En este caso se devuelve una COPIA.
array_par = array[mask]

print(array_par)

print()

# Modificamos la copia.
array_par[0] = -10

# El arreglo original no cambia.
print(array)

# Solo cambia la copia.
print(array_par)
```

```
[-3 -2 -1  0  1  2  3]
```

```
[False True False True False True False]
```

```
[-2  0  2]
```

```
[-3 -2 -1  0  1  2  3]
```

```
[-10  0  2]
```

6.7.5 Acceso a múltiples elementos mediante índices

Además del **slicing**, NumPy permite seleccionar múltiples filas o columnas indicando explícitamente los índices que se desean obtener.

Esta técnica es conocida como **Fancy Indexing**.

```
[43]: # =====  
# Fancy Indexing (índices múltiples)  
# =====  
  
# Creamos una matriz de 4x4.  
array = np.arange(16).reshape(4, 4)  
  
# Mostramos la matriz.  
print(array)  
  
print()  
  
# Seleccionamos las filas 1 y 2.  
print(array[[1, 2]])  
  
print()  
  
# Seleccionamos las columnas 1 y 2.  
print(array[:, [1, 2]])  
  
print()  
  
# Seleccionamos filas y columnas específicas.  
print(array[[1, 2], :][:, [1, 2]])  
  
print()  
  
# Forma equivalente utilizando np.ix().  
print(array[np.ix_([1, 2], [1, 2])])  
  
print()  
  
# Fancy Indexing devuelve una copia.  
ar = array[[2, 3, 1]]
```

```

print(ar)

print()

# Modificamos la copia.
ar[0] = 999

# Mostramos ambos arreglos.
print(ar)

print()

# La matriz original permanece sin cambios.
print(array)

```

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]

```

```

[[ 4  5  6  7]
 [ 8  9 10 11]]

```

```

[[ 1  2]
 [ 5  6]
 [ 9 10]
 [13 14]]

```

```

[[ 5  6]
 [ 9 10]]

```

```

[[ 5  6]
 [ 9 10]]

```

```

[[ 8  9 10 11]
 [12 13 14 15]
 [ 4  5  6  7]]

```

```

[[999 999 999 999]
 [ 12 13 14 15]
 [ 4  5  6  7]]

```

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]

```

7 3 Funciones matemáticas

Una de las principales ventajas de NumPy es que permite realizar operaciones matemáticas de forma **vectorizada**, es decir, aplicando una operación sobre todos los elementos de un arreglo sin necesidad de utilizar ciclos.

Estas operaciones pueden realizarse entre:

- Escalares y arreglos.
- Dos arreglos del mismo tamaño.
- Arreglos compatibles mediante **broadcasting**.

7.0.1 Operaciones con listas de Python

Antes de utilizar NumPy, es importante observar cómo se comportan las listas tradicionales de Python al realizar operaciones con escalares y otras listas.

```
[44]: # =====  
# Operaciones con listas de Python  
# =====  
  
# Creamos una lista.  
lista = [0, 1, 2, 3]  
  
# Multiplicar una lista no multiplica sus valores,  
# sino que repite su contenido.  
lista *= 3  
  
print(lista)  
  
print()  
  
# Agregar un elemento utilizando +=.  
lista += [1]  
  
print(lista)
```

```
[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]
```

```
[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 1]
```

7.0.2 Conclusión

Las máscaras booleanas y el **Fancy Indexing** permiten acceder de forma flexible a conjuntos específicos de datos dentro de un arreglo. Además, NumPy simplifica enormemente las operaciones matemáticas gracias a la vectorización, ofreciendo un rendimiento muy superior al de las listas tradicionales de Python.

7.1 Operaciones matemáticas con ndarrays

A diferencia de las listas tradicionales de Python, los **ndarrays** permiten realizar operaciones matemáticas directamente sobre todos sus elementos.

Estas operaciones se aplican de forma **elemento a elemento**, lo que simplifica el código y mejora considerablemente el rendimiento.

```
[45]: # =====  
# Operaciones entre un escalar y un ndarray  
# =====  
  
# Creamos un arreglo de cuatro elementos con valor 1.  
# Se utiliza el tipo de dato entero de 8 bits.  
array = np.ones(4, dtype=np.int8)  
  
# Mostramos el arreglo original.  
print(array)  
  
print()  
  
# Multiplicamos todos los elementos por 2.  
array *= 2  
  
print(array)  
  
print()  
  
# Sumamos 10 a cada elemento.  
array += 10  
  
print(array)  
  
print()  
  
# Mostramos el tipo de dato del arreglo.  
print(array.dtype)
```

```
[1 1 1 1]
```

```
[2 2 2 2]
```

```
[12 12 12 12]
```

```
int8
```


7.1.1 Operaciones entre arreglos

Cuando dos **ndarrays** poseen las mismas dimensiones, NumPy realiza las operaciones **elemento a elemento**.

Esto significa que cada posición de un arreglo opera con la posición correspondiente del otro arreglo.

```
[46]: # =====  
# Operaciones entre dos ndarrays  
# =====  
  
# Creamos el primer arreglo con todos sus valores iguales a 2.  
array_1 = np.ones(4) * 2  
  
# Creamos el segundo arreglo con valores consecutivos.  
array_2 = np.arange(4) + 1  
  
# Mostramos ambos arreglos.  
print(array_1)  
print(array_2)  
  
print()  
  
# Suma elemento a elemento.  
print(array_1 + array_2)  
  
print()  
  
# Resta elemento a elemento.  
print(array_1 - array_2)  
  
print()  
  
# Multiplicación elemento a elemento.  
print(array_1 * array_2)  
  
print()  
  
# División elemento a elemento.  
print(array_2 / array_1)
```

```
[2. 2. 2. 2.]
```

```
[1 2 3 4]
```

```
[3. 4. 5. 6.]
```

```
[ 1.  0. -1. -2.]
```

```
[2. 4. 6. 8.]
```

```
[0.5 1.  1.5 2. ]
```

7.2 3.1 Broadcasting

En ocasiones los arreglos que participan en una operación **no tienen las mismas dimensiones**.

NumPy resuelve este problema mediante un mecanismo denominado **Broadcasting**, que consiste en expandir automáticamente el arreglo más pequeño para hacerlo compatible con el arreglo de mayor tamaño.

Gracias a este comportamiento es posible realizar operaciones entre arreglos de diferentes dimensiones sin necesidad de crear copias adicionales de los datos.

7.2.1 ¿Qué ventajas ofrece el Broadcasting?

- Evita crear arreglos temporales innecesarios.
- Reduce el consumo de memoria.
- Simplifica considerablemente el código.
- Permite realizar operaciones vectorizadas de forma muy eficiente.

7.3 3.2 Operadores unarios y binarios

NumPy incorpora una gran cantidad de funciones matemáticas que permiten realizar operaciones de manera eficiente sobre todos los elementos de un **ndarray**.

Estas funciones se clasifican en:

- **Operadores unarios:** actúan sobre un único arreglo.
- **Operadores binarios:** realizan operaciones entre dos arreglos.

7.3.1 Operadores unarios

Operación	Descripción
<code>abs</code>	Valor absoluto de cada elemento.
<code>sqrt</code>	Calcula la raíz cuadrada de cada elemento.
<code>exp</code>	Calcula (e^x) para cada elemento.
<code>log</code> , <code>log10</code> , <code>log2</code>	Calcula logaritmos en distintas bases.
<code>sign</code>	Devuelve el signo de cada elemento (-1, 0 o 1).
<code>ceil</code>	Redondea hacia arriba.
<code>floor</code>	Redondea hacia abajo.
<code>isnan</code>	Indica si un elemento es NaN.
<code>cos</code> , <code>sin</code> , <code>tan</code>	Funciones trigonométricas.
<code>arccos</code> , <code>arcsin</code> , <code>arctan</code>	Funciones trigonométricas inversas.

7.3.2 Operadores binarios

Operación	Descripción
add	Suma dos arreglos.
subtract	Resta dos arreglos.
multiply	Multiplica dos arreglos elemento a elemento.
divide	Divide dos arreglos elemento a elemento.
maximum, minimum	Obtiene el mayor o menor valor entre dos elementos correspondientes.
equal, not_equal	Compara igualdad o desigualdad entre dos arreglos.
greater, greater_equal	Compara si un elemento es mayor o mayor o igual que otro.
less, less_equal	Compara si un elemento es menor o menor o igual que otro.

7.3.3 Ejemplo de operador unario

En el siguiente ejemplo se utiliza la función `sqrt()`, que calcula la raíz cuadrada de cada elemento del arreglo.

```
[47]: # =====
# Ejemplo de un operador unario
# =====

# Creamos un arreglo con valores del 0 al 4.
array = np.arange(5)

# Mostramos el arreglo original.
print(array)

print()

# Calculamos la raíz cuadrada de cada elemento.
print(np.sqrt(array))
```

```
[0 1 2 3 4]
```

```
[0.          1.          1.41421356  1.73205081  2.          ]
```

7.3.4 Ejemplo de operadores binarios

Las funciones binarias comparan o realizan operaciones entre dos arreglos del mismo tamaño, trabajando elemento por elemento.

```
[48]: # =====
# Ejemplo de operadores binarios
# =====

# Creamos dos arreglos.
array1 = np.arange(4)
```

```

array2 = np.array([0, -1, 2, -3])

# Mostramos ambos arreglos.
print(array1)
print(array2)

print()

# Comparamos si los elementos de array1 son mayores.
print(np.greater(array1, array2))

print()

# Comparamos si los elementos de array2 son menores.
print(np.less(array2, array1))

```

```

[0 1 2 3]
[ 0 -1  2 -3]

```

```
[False  True False  True]
```

```
[False  True False  True]
```

7.3.5 Conclusión

Los operadores unarios y binarios de NumPy permiten realizar cálculos y comparaciones de forma vectorizada sobre los elementos de un arreglo. Gracias a estas funciones es posible escribir código más limpio, eficiente y con un mejor rendimiento que utilizando ciclos tradicionales de Python.

8 4 Estadística descriptiva

La **estadística descriptiva** permite resumir y analizar un conjunto de datos mediante diferentes medidas que facilitan su interpretación.

Con NumPy es posible obtener información como:

- Valores máximos y mínimos.
- Promedio de los datos.
- Desviación estándar.
- Sumas y productos acumulados.
- Valores únicos.
- Operaciones entre conjuntos de datos.

Estas funciones son ampliamente utilizadas en análisis de datos, ciencia de datos e inteligencia artificial.

8.1 Funciones de estadística descriptiva

Función	Descripción
<code>sum(arr)</code>	Calcula la suma de todos los elementos del arreglo.
<code>mean(arr)</code>	Calcula la media aritmética de los elementos.
<code>std(arr)</code>	Calcula la desviación estándar.
<code>cumsum(arr)</code>	Devuelve un arreglo con la suma acumulada de los elementos.
<code>cumprod(arr)</code>	Devuelve un arreglo con el producto acumulado de los elementos.
<code>min(arr)</code>	Obtiene el valor mínimo del arreglo.
<code>max(arr)</code>	Obtiene el valor máximo del arreglo.
<code>any(arr)</code>	Devuelve True si al menos un elemento es verdadero.
<code>all(arr)</code>	Devuelve True si todos los elementos son verdaderos.
<code>unique(arr)</code>	Devuelve un arreglo con los valores únicos.
<code>in1d(arr1, arr2)</code>	Indica si cada elemento de <code>arr1</code> pertenece a <code>arr2</code> .
<code>union1d(arr1, arr2)</code>	Devuelve la unión de dos arreglos.
<code>intersect1d(arr1, arr2)</code>	Devuelve la intersección entre dos arreglos.

8.1.1 ¿Por qué es importante la estadística descriptiva?

Antes de aplicar algoritmos de análisis o aprendizaje automático, es recomendable conocer las características de los datos.

La estadística descriptiva permite detectar valores atípicos, comprender la distribución de los datos y obtener información útil para tomar decisiones durante el análisis.

8.1.2 Resumen

Las funciones estadísticas de NumPy permiten analizar grandes volúmenes de datos de forma rápida y eficiente. Gracias a ellas es posible obtener medidas descriptivas fundamentales sin necesidad de implementar algoritmos complejos, facilitando el análisis exploratorio de datos.

8.2 Suma y suma acumulada

NumPy ofrece funciones para obtener tanto la suma total de los elementos como la suma acumulada.

- `sum()` devuelve un único valor con la suma de todos los elementos.
- `cumsum()` devuelve un nuevo arreglo donde cada posición contiene la suma acumulada hasta ese elemento.

```
[49]: # =====
# Diferencia entre sum() y cumsum()
# =====

# Creamos un arreglo con valores del 0 al 8.
array1 = np.arange(9)

# Mostramos el arreglo.
print(array1)
```

```

print()

# Calculamos la suma total de todos los elementos.
print(array1.sum())

print()

# Calculamos la suma acumulada.
print(array1.cumsum())

```

```
[0 1 2 3 4 5 6 7 8]
```

```
36
```

```
[ 0  1  3  6 10 15 21 28 36]
```

8.3 Funciones `any()` y `all()`

Cuando un arreglo contiene valores booleanos (o valores numéricos interpretables como booleanos), NumPy dispone de dos funciones muy útiles:

- `any()` devuelve **True** si al menos un elemento es verdadero o diferente de cero.
- `all()` devuelve **True** únicamente cuando todos los elementos son verdaderos o diferentes de cero.

```

[50]: # =====
# Uso de any() y all()
# =====

# Creamos una matriz con valores negativos, positivos y cero.
array1 = np.arange(-8, 2).reshape(2, 5)

# Mostramos la matriz.
print(array1)

print()

# Comprueba si existe al menos un elemento distinto de cero.
print(array1.any())

print()

# Comprueba si todos los elementos son distintos de cero.
print(array1.all())

```

```

[[-8 -7 -6 -5 -4]
 [-3 -2 -1  0  1]]

```

True

False

8.4 Uso del parámetro **axis**

La mayoría de las funciones estadísticas de NumPy permiten utilizar el parámetro **axis**, el cual indica sobre qué dimensión se realizará la operación.

Los valores más utilizados son:

- **axis** = 0 → opera por columnas.
- **axis** = 1 → opera por filas.
- **axis** = 2 → opera sobre la tercera dimensión (profundidad), en arreglos tridimensionales.

```
[51]: # =====  
# Uso del parámetro axis  
# =====  
  
# Creamos una matriz de 2 filas y 5 columnas.  
array1 = np.arange(10).reshape(2, 5)  
  
# Mostramos la matriz.  
print(array1)  
  
print()  
  
# Evalúa cada columna por separado.  
print(array1.all(axis=0))  
  
print()  
  
# Evalúa cada fila por separado.  
print(array1.all(axis=1))  
  
print()  
  
# Evalúa toda la matriz.  
print(array1.all())
```

```
[[0 1 2 3 4]  
 [5 6 7 8 9]]
```

```
[False True True True True]
```

```
[False True]
```

False

8.4.1 Conclusión

Las funciones estadísticas de NumPy permiten analizar rápidamente un conjunto de datos. Además, el parámetro **axis** proporciona un mayor control sobre las operaciones, permitiendo realizarlas por filas, columnas o sobre otras dimensiones del arreglo según sea necesario.

9 5 Álgebra lineal

NumPy incorpora el módulo **numpy.linalg**, que proporciona un conjunto de herramientas para realizar operaciones de **álgebra lineal** de forma rápida y eficiente.

Estas funciones son ampliamente utilizadas en áreas como:

- Ciencia de datos.
- Inteligencia Artificial.
- Machine Learning.
- Ingeniería.
- Simulación numérica.

Algunas de las operaciones disponibles son el producto de matrices, cálculo de determinantes, matrices inversas, resolución de sistemas de ecuaciones y factorizaciones.

9.1 Funciones principales de **numpy.linalg**

Función	Descripción
<code>dot(mat1, mat2)</code>	Calcula el producto escalar entre dos arreglos. Si ambos son matrices bidimensionales, equivale a su multiplicación.
<code>matmul(mat1, mat2)</code>	Multiplica dos matrices.
<code>trace(mat)</code>	Calcula la suma de los elementos de la diagonal principal de una matriz.
<code>det(mat)</code>	Calcula el determinante de una matriz cuadrada.
<code>eig(mat)</code>	Obtiene los autovalores y autovectores de una matriz cuadrada.
<code>inv(mat)</code>	Calcula la matriz inversa.
<code>qr(mat)</code>	Realiza la factorización QR de una matriz.
<code>solve(A, b)</code>	Resuelve un sistema de ecuaciones lineales de la forma $\mathbf{Ax} = \mathbf{b}$.
<code>transpose(mat)</code>	Devuelve la matriz transpuesta.

9.1.1 Multiplicación de matrices

Existen varias formas de multiplicar matrices en NumPy:

- `np.matmul()`
- `np.dot()`
- Operador `@`

Cuando se trabaja con matrices bidimensionales, las tres alternativas producen el mismo resultado.


```
[52]: # =====
# Multiplicación de matrices
# =====

# Creamos una matriz de 4 filas por 3 columnas.
A = np.arange(12).reshape(4, 3)

# Creamos una matriz de 3 filas por 2 columnas.
B = np.arange(6).reshape(3, 2)

# Mostramos ambas matrices.
print("Matriz A")
print(A)

print()

print("Matriz B")
print(B)

print()

# Multiplicación utilizando np.matmul().
print("Resultado con np.matmul():")
print(np.matmul(A, B))

print()

# Multiplicación utilizando np.dot().
print("Resultado con np.dot():")
print(np.dot(A, B))

print()

# Multiplicación utilizando el operador @.
print("Resultado con el operador @:")
print(A @ B)
```

Matriz A

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

Matriz B

```
[[0 1]
 [2 3]
 [4 5]]
```

Resultado con `np.matmul()`:

```
[[10 13]
 [28 40]
 [46 67]
 [64 94]]
```

Resultado con `np.dot()`:

```
[[10 13]
 [28 40]
 [46 67]
 [64 94]]
```

Resultado con el operador `@`:

```
[[10 13]
 [28 40]
 [46 67]
 [64 94]]
```

9.1.2 Conclusión

NumPy ofrece múltiples formas de realizar operaciones de álgebra lineal. En el caso de matrices bidimensionales, `np.matmul()`, `np.dot()` y el operador `@` producen el mismo resultado, permitiendo escoger la sintaxis que resulte más cómoda y legible para cada situación.

9.2 Resolver un sistema de ecuaciones lineales

Una de las funciones más utilizadas del módulo `numpy.linalg` es `solve()`, la cual permite resolver sistemas de ecuaciones lineales de la forma:

[$Ax=b$]

donde:

- **A** representa la matriz de coeficientes.
- **x** representa el vector de incógnitas.
- **b** representa el vector de resultados.

En el siguiente ejemplo se resolverá un sistema de tres ecuaciones con tres incógnitas.

```
[53]: # =====
# Resolver un sistema de ecuaciones lineales
# =====

# Importamos el módulo de álgebra lineal.
import numpy.linalg as lg
import math

# Sistema de ecuaciones:
#
#  $3x + 2y + 5z = 12$ 
```

```

#  $x - 3y - z = 2$ 
#  $2x - y + 12z = 32$ 

# Creamos la matriz de coeficientes.
A = np.array([
    [3, 2, 5],
    [1, -3, -1],
    [2, -1, 12]
])

# Creamos el vector de resultados.
b = np.array([12, 2, 32])

# Mostramos los datos del sistema.
print("Matriz A:")
print(A)

print()

print("Vector b:")
print(b)

print()

# Resolvemos el sistema  $Ax = b$ .
x, y, z = lg.solve(A, b)

# Mostramos los valores obtenidos.
print(f"x = {x}")
print(f"y = {y}")
print(f"z = {z}")

print()

# Verificamos que la solución satisfice
# las tres ecuaciones originales.
print(math.isclose(3 * x + 2 * y + 5 * z, 12))
print(math.isclose(x - 3 * y - z, 2))
print(math.isclose(2 * x - y + 12 * z, 32))

```

Matriz A:

```

[[ 3  2  5]
 [ 1 -3 -1]
 [ 2 -1 12]]

```

Vector b:

```

[12  2 32]

```

```
x = 0.7543859649122807
y = -1.2280701754385965
z = 2.43859649122807
```

```
True
True
True
```

9.2.1 Matriz identidad

La **matriz identidad** es una matriz cuadrada cuyos elementos de la diagonal principal son iguales a **1** y el resto de los elementos son **0**.

Al multiplicar una matriz por la matriz identidad, el resultado es la misma matriz.

```
[54]: # =====
# Matriz identidad
# =====

# Creamos una matriz identidad de tamaño 4x4.
I = np.identity(4)

print("Matriz identidad:")
print(I)

print()

# Creamos una matriz de ejemplo.
A = np.arange(16).reshape(4, 4)

print("Matriz A:")
print(A)

print()

# Multiplicar una matriz por la identidad
# devuelve la misma matriz.
print("A × I =")
print(np.matmul(A, I))
```

Matriz identidad:

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

Matriz A:

```
[[ 0  1  2  3]
 [ 4  5  6  7]]
```

```
[ 8  9 10 11]
[12 13 14 15]]
```

```
A × I =
[[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]
 [12. 13. 14. 15.]]
```

9.2.2 Conclusión

El módulo `numpy.linalg` proporciona herramientas muy útiles para resolver problemas de álgebra lineal. Con funciones como `solve()` es posible resolver sistemas de ecuaciones de forma sencilla, mientras que la **matriz identidad** permite comprender una de las propiedades fundamentales de la multiplicación de matrices.

10 6 Filtrado de datos

NumPy proporciona la función `np.where()`, la cual permite seleccionar o reemplazar valores de un arreglo según una condición lógica.

Su sintaxis general es:

```
np.where(condición, valor_si_verdadero, valor_si_falso)
```

Esta función es ampliamente utilizada en análisis y limpieza de datos, ya que evita el uso de ciclos y permite realizar operaciones de forma vectorizada.

10.1 Reemplazar valores mediante una condición

En este ejemplo se reemplazan todos los precios menores que **1.00** por el valor **1.00** utilizando la función `np.where()`.

```
[55]: # =====
# Reemplazar valores utilizando np.where()
# =====

# Creamos un arreglo con diferentes precios.
prices = np.array([0.99, 14.49, 19.99, 20.99, 0.49])

# Mostramos únicamente los precios menores a 1.
print(prices[prices < 1])

print()

# np.where() devuelve 1.0 cuando la condición se cumple;
# en caso contrario conserva el valor original.
print(np.where(prices < 1, 1.0, prices))
```

```
[0.99 0.49]
```

```
[ 1.    14.49 19.99 20.99  1.   ]
```

10.2 Sustituir datos fuera de rango

Una aplicación frecuente de `np.where()` consiste en limpiar datos incorrectos.

En este ejemplo, todos los valores negativos se reemplazan por **0**.

```
[56]: # =====  
# Reemplazar valores negativos  
# =====  
  
# Creamos una matriz con valores positivos y negativos.  
bank_transfers_values = np.array([  
    0.99,  
    -1.49,  
    19.99,  
    20.99,  
    -0.49,  
    12.1  
]).reshape(2, 3)  
  
# Mostramos la matriz original.  
print(bank_transfers_values)  
  
print()  
  
# Reemplazamos los valores negativos por cero.  
clean_data = np.where(bank_transfers_values > 0,  
                      bank_transfers_values,  
                      0)  
  
# Mostramos el resultado.  
print(clean_data)
```

```
[[ 0.99 -1.49 19.99]  
 [20.99 -0.49 12.1 ]]
```

```
[[ 0.99  0.   19.99]  
 [20.99  0.   12.1 ]]
```

10.3 Reemplazar valores NaN

Los valores **NaN** (*Not a Number*) representan datos faltantes o inválidos.

Con `np.isnan()` es posible identificarlos y sustituirlos utilizando `np.where()`.

```
[57]: # =====
# Reemplazar valores NaN
# =====

# Creamos una lista con un valor NaN.
data = [10, 12, -143, np.nan, 1, -3]

# Sustituimos NaN por cero.
data_clean = np.where(np.isnan(data), 0, data)

# Mostramos los resultados.
print(data_clean)

print()

# Comparamos las direcciones de memoria.
print(id(data))
print(id(data_clean))
```

```
[ 10.   12. -143.    0.    1.   -3.]
```

```
2133599362304
```

```
2133599519728
```

10.4 Obtener los índices que cumplen una condición

Cuando **np.where()** recibe únicamente una condición, devuelve los índices de los elementos que la cumplen.

```
[58]: # =====
# Obtener índices con np.where()
# =====

# Creamos una matriz de ejemplo.
bank_transfers_values = np.array([
    0.99,
    -1.49,
    19.99,
    20.99,
    -0.49,
    12.1
]).reshape(2, 3)

# Mostramos la matriz.
print(bank_transfers_values)

print()
```

```

# Obtenemos los índices de los valores positivos.
credits = np.where(bank_transfers_values > 0)

print(credits)

print()

# Mostramos los valores utilizando los índices obtenidos.
print(bank_transfers_values[credits])

print()

# Forma equivalente utilizando una máscara booleana.
print(bank_transfers_values[bank_transfers_values > 0])

```

```

[[ 0.99 -1.49 19.99]
 [20.99 -0.49 12.1 ]]

```

```

(array([0, 0, 1, 1]), array([0, 2, 0, 2]))

```

```

[ 0.99 19.99 20.99 12.1 ]

```

```

[ 0.99 19.99 20.99 12.1 ]

```

10.5 Seleccionar valores entre dos arreglos

También es posible utilizar `np.where()` para escoger valores provenientes de dos arreglos distintos según una condición.

```

[59]: # =====
# Seleccionar valores entre dos arreglos
# =====

# Recompensas por defecto.
rewards_default = np.arange(6)

print(rewards_default)

print()

# Recompensas mejoradas.
rewards_upgrade = np.arange(6) * 10

print(rewards_upgrade)

print()

# Puntos obtenidos por cada usuario.

```



```

daily_points = np.array([0, 0, 1, 6, 0, 2])

print(daily_points)

print()

# Si los puntos son mayores que 1,
# se utiliza la recompensa mejorada;
# de lo contrario, la recompensa por defecto.
final_rewards = np.where(
    daily_points > 1,
    rewards_upgrade,
    rewards_default
)

print(final_rewards)

```

```
[0 1 2 3 4 5]
```

```
[ 0 10 20 30 40 50]
```

```
[0 0 1 6 0 2]
```

```
[ 0  1  2 30  4 50]
```

10.5.1 Conclusión

La función `np.where()` es una herramienta muy útil para realizar operaciones de filtrado y limpieza de datos. Permite reemplazar valores, localizar elementos que cumplen determinadas condiciones y combinar arreglos de forma eficiente, siendo una de las funciones más utilizadas en análisis de datos y Machine Learning.

11 7 Números aleatorios

Python ofrece diferentes herramientas para generar números aleatorios.

Las más utilizadas son:

- El módulo **random**, incluido en la biblioteca estándar de Python.
- El módulo **numpy.random**, diseñado para trabajar con arreglos y operaciones numéricas de forma eficiente.

Estas herramientas son ampliamente utilizadas en simulaciones, estadística, inteligencia artificial y ciencia de datos.

11.1 Módulo **random**

El módulo **random** permite generar números aleatorios y seleccionar elementos al azar de una colección.

```
[60]: # =====
# Uso del módulo random
# =====

# Importamos la librería.
import random

# Número decimal aleatorio entre 0 y 1.
print(random.random())

# Número entero entre 0 y 5.
print(random.randint(0, 5))

# Número decimal entre 0 y 5.
print(random.uniform(0, 5))
```

0.6435868292561997

5

1.0924573641443025

11.1.1 Seleccionar un elemento aleatorio

La función **choice()** permite escoger un elemento al azar de una lista.

```
[61]: # =====
# Elegir un elemento aleatorio
# =====

# Lista de ejemplo.
names = ["Pikachu", "Eevee", "Charmander"]

# Seleccionamos un elemento al azar.
print(random.choice(names))
```

Charmander

11.2 7.1 NumPy Random

El módulo **numpy.random** facilita la generación de arreglos completos con valores aleatorios, siendo una herramienta fundamental para el análisis de datos y Machine Learning.

```
[62]: # =====
# Generar números aleatorios con NumPy
# =====

# Número decimal entre 0 y 1.
print(np.random.rand())

# Número entero entre 0 y 5.
```

```
print(np.random.randint(0, 5))

# Matriz de números enteros aleatorios.
print(np.random.randint(5, size=(2, 4)))
```

```
0.4269338998610902
```

```
2
```

```
[[3 4 4 4]
 [1 3 1 1]]
```

11.2.1 Distribución normal

La función `randn()` genera números aleatorios siguiendo una distribución normal (gaussiana), con media igual a **0** y desviación estándar igual a **1**.

```
[63]: # =====
# Distribución normal
# =====

# Generamos una matriz de valores aleatorios
# con distribución normal.
print(np.random.randn(4, 2))
```

```
[[ -0.21337296  0.76705981]
 [ -0.84539817  1.05202728]
 [ -1.71531491 -1.150197  ]
 [  0.78459732  1.2277346  ]]
```

11.2.2 Otras distribuciones

NumPy permite generar números aleatorios utilizando diferentes distribuciones estadísticas según las necesidades del problema.

```
[64]: # =====
# Otras distribuciones estadísticas
# =====

# Distribución binomial.
print(np.random.binomial(n=5, p=0.3))

# Distribución uniforme.
print(np.random.uniform(low=0, high=10))

# Distribución de Poisson.
print(np.random.poisson(lam=2, size=(2, 2)))
```

```
2
```

```
1.2601465287856306
```

```
[[1 2]
 [1 1]]
```

11.3 7.2 Random Seed

Los números generados por un computador son realmente **pseudoaleatorios**, ya que se obtienen mediante algoritmos matemáticos.

La función `seed()` establece un valor inicial para el generador de números aleatorios, permitiendo reproducir exactamente los mismos resultados en diferentes ejecuciones del programa.

```
[65]: # =====  
# Misma semilla (seed)  
# =====  
  
# Utilizamos siempre la misma semilla.  
# Esto genera exactamente la misma secuencia  
# de números aleatorios.  
  
seed = 5  
  
for _ in range(5):  
    np.random.seed(seed)  
  
    print(np.random.rand())  
    print(np.random.rand())  
    print()
```

```
0.22199317108973948  
0.8707323061773764
```

```
0.22199317108973948  
0.8707323061773764
```

```
0.22199317108973948  
0.8707323061773764
```

```
0.22199317108973948  
0.8707323061773764
```

```
0.22199317108973948  
0.8707323061773764
```

```
[66]: # =====  
# Semillas diferentes  
# =====  
  
# Ahora utilizamos una semilla distinta  
# en cada iteración.
```

```
for seed in range(6):

    np.random.seed(seed)

    print(np.random.rand())
    print(np.random.rand())
```

```
0.5488135039273248
0.7151893663724195
0.417022004702574
0.7203244934421581
0.43599490214200376
0.025926231827891333
0.5507979025745755
0.7081478226181048
0.9670298390136767
0.5472322491757223
0.22199317108973948
0.8707323061773764
```

11.3.1 Conclusión

La generación de números aleatorios es una herramienta esencial en programación científica. Python ofrece el módulo **random** para tareas generales, mientras que **numpy.random** proporciona funciones más avanzadas y eficientes para trabajar con arreglos y distribuciones estadísticas. Además, el uso de una **semilla (seed)** permite obtener resultados reproducibles, aspecto fundamental en experimentos y proyectos de ciencia de datos.

12 8 Escritura y lectura de ndarrays a archivos

NumPy permite almacenar arreglos directamente en archivos para utilizarlos posteriormente o compartirlos con otros programas.

Los formatos más utilizados son:

- **.npy**: almacena un único arreglo de NumPy.
- **.npz**: almacena varios arreglos dentro de un mismo archivo.

Estos formatos permiten guardar la información de forma eficiente y mantener el tipo de dato original.

[]:

12.1 Guardar un arreglo en un archivo

La función **np.save()** permite almacenar un arreglo en un archivo con extensión **.npy**.

```
[67]: # =====
      # Guardar un ndarray en un archivo
      # =====
```

```

# Importamos el módulo para trabajar con rutas.
import os

# Construimos la ruta del archivo.
ruta = os.path.join("res", "o_values_array.npy")

# Generamos una matriz aleatoria.
values = np.random.randint(9, size=(3, 3))

# Guardamos la matriz en un archivo .npy.
np.save(ruta, values)

# Mostramos la matriz almacenada.
print(values)

```

```

[[6 6 0]
 [8 4 7]
 [0 0 7]]

```

12.2 Leer un archivo .npy

La función `np.load()` permite recuperar un arreglo previamente almacenado.

```

[68]: # =====
# Leer un ndarray desde un archivo
# =====

# Cargamos el arreglo almacenado.
old_values = np.load(ruta)

# Mostramos su contenido.
print(old_values)

```

```

[[6 6 0]
 [8 4 7]
 [0 0 7]]

```

12.3 Guardar varios arreglos en un mismo archivo

Cuando se desea almacenar más de un arreglo, se utiliza la función `np.savez()`, la cual genera un archivo con extensión `.npz`.

```

[69]: # =====
# Guardar múltiples ndarrays
# =====

# Ruta del archivo.
ruta = os.path.join("res", "o_array_group.npz")

```

```

# Creamos varios arreglos.
values1 = np.random.randint(9, size=(3, 3))
values2 = np.random.randint(4, size=(2, 2))
values3 = np.random.randint(16, size=(4, 4))

# Guardamos todos los arreglos.
np.savez(ruta, values1, values2, values3)

```

12.4 Leer un archivo .npz

Al cargar un archivo **.npz**, NumPy devuelve un objeto que contiene todos los arreglos almacenados.

```

[70]: # =====
# Leer múltiples ndarrays
# =====

# Abrimos el archivo.
npzfile = np.load(ruta)

# Mostramos el tipo de objeto.
print(type(npzfile.files))

# Mostramos los nombres de los arreglos.
print(npzfile.files)

print()

# Recorremos cada arreglo almacenado.
for key in npzfile.files:

    print(f"{key}:")
    print(npzfile[key])
    print()

```

```

<class 'list'>
['arr_0', 'arr_1', 'arr_2']

```

```

arr_0:
[[1 5 7]
 [0 1 4]
 [6 2 1]]

```

```

arr_1:
[[2 3]
 [0 2]]

```

```

arr_2:

```

```
[[ 5 10  0  0]
 [ 4 14  4  9]
 [ 3 11 15  2]
 [ 4  6  9  3]]
```

12.5 8.1 ¿Por qué utilizar archivos .npz?

Muchas bases de datos almacenan la información en archivos **CSV** o **TXT**, los cuales deben leerse y convertirse nuevamente a arreglos antes de poder utilizarlos.

Guardar los datos en formato **.npz** evita ese proceso de conversión y permite cargar la información de forma mucho más rápida.

```
[71]: # =====
# Convertir un archivo CSV a formato .npz
# =====

import os

# Ruta del archivo CSV.
ruta_csv = os.path.join("res", "fdata.csv")

# Leemos el contenido del archivo.
with open(ruta_csv, "r") as f:

    data_str = f.read()

# Convertimos el texto en una lista.
data = data_str.split(",")

# Convertimos la lista en un ndarray.
data_array = np.array(data, dtype=np.int8).reshape(1000, 1000)

# Ruta del nuevo archivo.
ruta_npz = os.path.join("res", "o_fdata.npz")

# Guardamos el arreglo.
np.save(ruta_npz, data_array)
```

12.6 Comparación de rendimiento

Jupyter ofrece la instrucción mágica **%timeit**, la cual permite medir el tiempo de ejecución de un bloque de código.

A continuación se compara el tiempo necesario para leer un archivo CSV frente a un archivo **.npz**.

```
[72]: %%timeit
```



```
# Leer y convertir un archivo CSV.

with open(ruta_csv, "r") as f:

    data_str = f.read()

data = data_str.split(",")

data_array = np.array(data, dtype=np.int8).reshape(1000, 1000)
```

368 ms ± 252 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

```
[73]: %%timeit

# Leer directamente un archivo .npy.

data = np.load(ruta_npy)
```

813 s ± 90.5 s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

```
[74]: %%timeit

# Leer un archivo CSV utilizando NumPy.

result = np.fromfile(
    ruta_csv,
    dtype=np.int8,
    sep=", "
)
```

460 ms ± 29.2 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

12.6.1 Velocidad frente a memoria

El formato **.npy** permite cargar arreglos mucho más rápido que un archivo CSV, ya que conserva directamente la estructura y el tipo de datos del arreglo.

Aunque requiere espacio adicional para almacenar el archivo generado, su uso mejora considerablemente el rendimiento cuando los datos se utilizan repetidamente.

12.7 Conclusión

NumPy proporciona formatos especializados para almacenar arreglos de manera eficiente. Los archivos **.npy** permiten guardar un único arreglo, mientras que los archivos **.npz** almacenan varios arreglos dentro de un mismo archivo. Estos formatos reducen el tiempo de carga de los datos y son ampliamente utilizados en proyectos de análisis de datos, inteligencia artificial y aprendizaje automático.

```
[ ]:
```